

Module 02 — Containerlab Basics

FOR DUMMIES

Goal: internalize the lab lifecycle — deploy, inspect, access, configure, save, destroy, restore — using the 2-node test topology. By the end, Containerlab should feel like “GNS3, but it’s code.”



Shortcut, once you know these: `./lab` at the repo root wraps every command in this module — `./lab deploy`, `./lab ssh`, `./lab graph`, `./lab destroy`, or run it bare for a menu. Type the raw commands below first; the point of this module is knowing what the wrapper does. After that, use the wrapper.

1. Read the topology file before deploying it

```
cd ~/clab-bootstrap
cat topologies/testlab.clab.yml
```

Three blocks, and this is most of Containerlab’s grammar already:

- `kinds`: — defaults per device type. `ceos` nodes get the image tag here so every node doesn’t repeat it.
- `nodes`: — the devices. Two spines, nothing more to say about them.
- `links`: — virtual cables. `["spine1:eth1", "spine2:eth1"]` is literally “plug a cable between these two ports.”

That’s the topology plane from module 00, in its entirety. No GUI, no dragging — which means it diffs, reviews, and version-controls like any other code.

2. Deploy, and read the output

```
./scripts/02-deploy-lab.sh topologies/testlab.clab.yml
```

While it runs (~60s — cEOS boots a whole EOS in there), notice what the log says it’s doing: creating a Docker network (`172.20.20.0/24` — the management network, every node’s Ma0 lands here), creating the lab directory, creating containers, wiring the link, writing `/etc/hosts` entries and SSH config. The final table shows each node’s management IP.

Look at what appeared next to the topology file:

```
ls topologies/clab-testlab/
```

That directory is the lab's persistent state — per-node startup-configs live in there. It's what makes destroy/deploy round-trips keep your work.

3. Get into the devices

```
ssh admin@clab-testlab-spine1      # password: admin
```

If hostname resolution fails (fresh shell, common): use the management IP from the table, or the always-works backdoor:

```
docker exec -it clab-testlab-spine1 Cli
```

Remember that `docker exec` trick — it works even after you've broken the management config and locked yourself out of SSH, which you will eventually do, on purpose or otherwise.

4. Configure the link, by hand

On spine1 — this is plain EOS, and if you know IOS you know 95% of it:

```
enable
configure terminal
interface Ethernet1
    no switchport
    ip address 10.10.0.0/31
    no shutdown
exit
write memory
```

On spine2, same thing with 10.10.0.1/31. Then verify:

```
show lldp neighbors      ! spine2 should appear on Et1
ping 10.10.0.0           ! from spine2 -> sub-millisecond replies
```

LLDP proving the virtual cable, ping proving L3 — the data plane works.

5. The lifecycle: destroy is not delete

This is the habit that separates Containerlab from GNS3 thinking:

```
# you already did write memory on both nodes, right?
./scripts/02-deploy-lab.sh topologies/testlab.clab.yml destroy
docker ps          # gone - the RAM is back
```

Now bring it back:

```
./scripts/02-deploy-lab.sh topologies/testlab.clab.yml
ssh admin@clab-testlab-spine1
show running-config interfaces Ethernet1  # your config survived
```

The containers were destroyed; the `clab-testlab/` directory wasn't, and the startup-configs in it boot right back. Labs are ephemeral; state is a directory. Want a genuinely fresh start someday? Destroy, delete `clab-testlab/`, deploy.

6. The observation toolkit

```
./scripts/02-deploy-lab.sh topologies/testlab.clab.yml inspect  # node table again
./scripts/02-deploy-lab.sh topologies/testlab.clab.yml graph    # http://<vm-ip>:50080
```

```
docker stats                                     # live RAM/CPU per node – watch this
docker logs clab-testlab-spine1                 # when a node won't boot
```

`docker stats` is the one to internalize: each cEOS node is 0.5–1 GB of RAM, the VM has a fixed budget, and module 03 doubles the node count.

7. Checkpoint

1. Break it: `shutdown` spine1's Ethernet1, watch the LLDP entry age out on spine2, `no shutdown`, watch it return.
2. Destroy the lab, redeploy, and confirm your IPs survived — *without* looking back at step 5.
3. Explain to a rubber duck what `clab-testlab/` is and why deleting it changes what the next deploy boots into.

Then destroy the testlab and leave it destroyed — module 03 needs the RAM.

What you learned

The full lab lifecycle, the management network vs. the data plane, the persistence model, and the escape hatch (`docker exec ... cli`). Also, quietly, the thing that makes the rest of the course work: the topology was code, so in module 03 a bigger topology is just a bigger file.

Next: [Module 03 — The spine-leaf fabric](#)